实验 3：算法优化实验指导

一、实验目标

1. 基于实验 2 训练的模型，掌握模型轻量化核心技术（量化、剪枝、蒸馏），实现模型在参数量和计算量上的有效缩减

2. 针对实验 1 的数据集特点（图像/文本/多模态），运用推理加速方法（计算图优化、硬件加速、并行调度）提升部署效率，优化推理全流程耗时

3. 深入分析量化误差与精度-速度-资源的权衡关系，掌握性能损失控制与精度稳定性保障策略

4. 实现模型在不同硬件资源约束下的高效适配，优化资源占用效率

5. 提升模型抗干扰能力，增强在实际部署场景中的鲁棒性

6. 形成 "性能 - 精度 - 资源 - 鲁棒性" 四维优化评估体系，培养工程化思维和综合优化能力

二、实验环境

硬件：

-基础配置：CPU（≥4 核）、GPU；

-异构设备：含低功耗 CPU 或边缘设备（可选），模拟实际部署场景。

软件：

- PyCharm（Python 3.12）、Anaconda（Python 3.12）

-核心框架：PyTorch 2.4+（训练加速与动态图优化）、ONNX 1.17+（计算图优化与跨框架转换）；

-加速工具：TensorRT 10.0+（GPU 推理加速）、OpenVINO 2025.1+（CPU 加速）、ONNX Runtime 1.20+（跨平台推理）；

-辅助库：torch.ao.quantization（量化工具）、torch.nn.utils.prune（剪枝工具）、pandas（结果分析，关联实验 1 数据）。

-选装/拓展：bitsandbytes、GPTQ/AWQ 实现库、NNCF（仅用于选做量化对比，不强制安装）

三、实验学时与安排

共 4 学时，分 2 个阶段完成

- 阶段 1：模型优化（2-3 学时） 基于实验 2 训练的模型从性能 - 精度 - 资源 - 鲁

棒性四个维度中至少一个维度进行优化；

- 阶段 2：优化与部署验证（1-2 学时）优化模型在 CPU/GPU 上的推理验证，用实验 1 测试集验证性能。（选做）

四、实验步骤

阶段 1：模型优化（3.1 模块）

目标：模型优化（2-3 学时） 基于实验 2 训练的模型从性能 - 精度 - 资源 - 鲁棒性四个维度中至少一个维度（下面 6 个任务至少一个）进行优化。

任务 3.1.1 模型量化实现

1. 量化方法选择：

- 针对实验 2 的 ResNet-18（CV 模型）：采用静态量化（需校准数据）；

- 针对实验 2 的 DistilBERT（NLP 模型）：采用动态量化（适合文本推理）。

-**【选做/拓展】**若实验 2 选用 Transformer 类或小型语言模型，可选做以下一种量化对比：GPTQ（W4A16）、AWQ、SmoothQuant（W8A8）、旋转量化（QuaRot/DuQuant）或 KV Cache INT8 量化。选做只需报告困惑度或分类准确率变化，不要求实现 QAT，不要求 FP4。

2. 量化实践：

3. 量化效果评估：

- 参数量对比：计算量化前后模型大小（如 FP32→INT8，模型体积减少 75%）；

- 精度对比：用实验 1 的测试集（./data/dataset/test.csv）评估 Top-1 准确率损失（目标≤2%）。

- 【选做/拓展】 若完成选做量化，可补充报告困惑度（PPL）或下游分类准确率变化，不改变基本评分要求。

任务 3.1.2 模型剪枝实现

1. 剪枝方法选择：

- 针对实验 2 的 ResNet-18（CV 模型）：采用结构化通道剪枝（保留网络结构规整性，便于部署），基于 L1_norm 准则筛选冗余卷积通道；

- 针对实验 2 的 DistilBERT（NLP 模型）：采用注意力头剪枝 + 神经元剪枝（聚焦 Transformer 核心结构），通过注意力权重方差筛选冗余头，通过激活值稀疏性筛选冗余神经元。

-【选做/拓展】 可在剪枝后微调之外，了解低秩分解（SVD）补偿思路；不要求实现联合量化-低秩压缩框架。

2. 剪枝实践：

- 预训练模型加载：加载实验 2 训练的 ResNet-18 和 DistilBERT 模型及权重；

- 剪枝阈值确定：通过验证集（./data/dataset/val.csv）测试不同剪枝比例（5%-30%）下的精度变化，确定目标剪枝比例（CV 模型通道保留率≥70%，NLP 模型注意力头保留率≥60%）；

- 剪枝执行：使用剪枝工具（如 TorchPrune）移除冗余结构，生成剪枝后模型；

- 微调恢复：在训练集上对剪枝后模型微调 3-5 个 epoch，恢复精度损失。

3. 剪枝效果评估：

- 计算量对比：统计剪枝前后模型的 FLOPs（浮点运算次数），CV 模型目标减少≥30%，NLP 模型目标减少≥25%；

- 参数量对比：计算剪枝前后模型参数量，目标减少比例与 FLOPs 匹配；

- 精度对比：用实验 1 的测试集（./data/dataset/test.csv）评估 Top-1 准确率（CV）/ 分类准确率（NLP），损失目标≤3%。

任务 3.1.3 模型蒸馏实现

1. 蒸馏策略选择：

- 针对实验 2 的 ResNet-18（CV 模型）：以 ResNet-50（更高精度的同架构模型）为教师，ResNet-18 为学生，采用 “软标签 + 硬标签” 混合蒸馏（温度系数 T=5-10）；

- 针对实验 2 的 DistilBERT（NLP 模型）：以 BERT-base（完整版本）为教师，DistilBERT 为学生，采用 “注意力蒸馏 + 隐藏层特征蒸馏”（保留 Transformer 核心知识）。

-【选做/拓展】 若实验 2 使用了 LoRA 微调，可选做将 LoRA 适配器蒸馏回紧凑学生模型，观察部署时是否减少额外参数；不要求实现复杂蒸馏损失。

2. 蒸馏实践：

- 教师模型准备：加载预训练的 ResNet-50（在实验 1 数据集上微调）和 BERT-base（在实验 1 文本数据集上微调）；

- 蒸馏损失设计：CV 模型损失 = 0.3× 硬标签交叉熵 + 0.7× 软标签 KL 散度；NLP

模型损失 = 0.2× 硬标签损失 + 0.5× 注意力映射 MSE+0.3× 隐藏层特征 MSE；

- 蒸馏训练：固定教师模型参数，以实验 2 的训练集训练学生模型，学习率为原训练的 1/5， epoch=10-15；

- 模型保存：保存蒸馏后的 ResNet-18 和 DistilBERT 模型。

3. 蒸馏效果评估：

- 推理速度对比：在相同硬件（CPU/GPU）上测试蒸馏前后模型的单样本推理时间，目标加速≥20%；

- 精度对比：测试集上的准确率，要求蒸馏后学生模型精度接近原模型（损失≤1%），且优于同量级未蒸馏模型；

- 模型大小对比：蒸馏后模型文件大小（与原模型相比），目标减少≥10%（结合轻量化效果）。

任务 3.1.4 推理加速优化实现

1. 加速方法选择：

- 针对 ResNet-18（CV 模型）：采用 TensorRT 优化（支持层融合、FP16 混合精度推理）；

- 针对 DistilBERT（NLP 模型）：采用 ONNX Runtime 优化（支持 Transformer 算子融合、动态批处理）。

-【选做/拓展】 对 Transformer 类模型，可选做启用 KV Cache 复用或 KV Cache INT8 压缩，观察首 token 延迟和内存变化；不要求边缘部署。

2. 加速实践：

- 模型格式转换：将 PyTorch 模型转换为 ONNX 格式（ResNet-18 需指定输入尺寸，DistilBERT 需指定序列长度）；

- 推理引擎配置：TensorRT 对 ResNet-18 启用 FP16 模式、层融合；ONNX Runtime 对 DistilBERT 启用 “ORTOptimizationLevel.ORT_ENABLE_ALL” 优化、设置动态批处理范围（1-32）；

- 推理 pipeline 构建：编写 Python 接口调用优化后的模型，实现输入预处理（图像 Resize / 文本 Tokenize）→推理→输出后处理的端到端流程。

3. 加速效果评估：

- 延迟对比：测试单样本推理延迟（ms）和批量推理延迟（批大小 32），目标较

原 PyTorch 推理降低≥40%；

- 吞吐量对比：计算单位时间（每秒）处理样本数，目标提升≥50%；

- 精度一致性：测试集上的准确率与原模型差异≤1%（确保加速不显著损失精度）。

任务 3.1.5 资源占用优化实现

1. 资源优化方法选择：

- 针对 ResNet-18（CV 模型）：采用权重稀疏存储（仅保存非零权重）+ 内存复用（推理时共享中间层缓存）；

- 针对 DistilBERT（NLP 模型）：采用嵌入层量化（词嵌入矩阵从 FP32→INT16）+ 动态权重加载（推理时按需加载 Transformer 层权重）。

2. 资源优化实践：

- 权重稀疏化：对 ResNet-18 的卷积层权重进行 L0 正则化训练（稀疏率 30%-50%），保存时仅存储非零值及其索引；

- 内存复用配置：修改推理代码，对特征图中间缓存设置固定内存块，循环复用（避免动态内存申请）；

- 嵌入层量化：对 DistilBERT 的词嵌入矩阵进行 INT16 量化（保留梯度微调），测试集精度损失≤0.5% 后固定；

- 动态加载实现：将 DistilBERT 的 Transformer 层权重拆分存储，推理时根据输入长度动态加载对应层。

3. 资源优化效果评估：

- 存储占用对比：模型文件存储大小（稀疏化 / 量化后），目标减少≥40%；

- 内存峰值对比：推理时 GPU/CPU 内存峰值占用，目标降低≥30%；

- 推理稳定性：连续推理 1000 次，检查是否出现内存泄漏（内存占用波动≤5%）。

任务 3.1.6 模型鲁棒性提升实现

1. 鲁棒性方法选择：

- 针对 ResNet-18（CV 模型）：采用对抗训练（PGD 攻击生成对抗样本，融入训练集）+ 数据增强（随机高斯噪声、随机裁剪扰动）；

- 针对 DistilBERT（NLP 模型）：采用文本扰动对抗训练（同义词替换、随机插入停用词）+ 模型正则化（隐藏层 dropout 率提高至 0.3）。

-【选做/拓展】 对语言模型可选做简单越狱提示测试或同义词替换扰动，观察压缩前后安全对齐是否削弱；不要求实现推理级对齐算法。

2. 鲁棒性优化实践：

- 对抗样本生成：对 CV 训练集，用 PGD 算法（ε=8/255，步长 = 2/255，迭代 10 次）生成对抗样本；对 NLP 训练集，用 TextFooler 工具生成扰动文本（扰动率≤15%）；

- 增强训练：将对抗样本与原始样本按 1:1 混合，以实验 2 的训练参数微调模型（epoch=5），CV 模型额外加入随机高斯噪声（σ=0.01）；

- 鲁棒性验证：在测试集上加入不同强度噪声（CV：高斯噪声 σ=0.02；NLP：10% 同义词替换），评估模型稳定性。

3. 鲁棒性效果评估：

- 干净数据精度：测试集原始数据上的准确率，确保与优化前差异≤1%；

- 抗干扰精度：带噪声测试集上的准确率，目标较优化前提升≥5%；

- 扰动容忍度：逐步提高噪声强度，记录模型准确率下降至 80% 时的扰动阈值（目标较优化前提高≥30%）。

阶段 2：优化与部署验证（3.2 模块）（选做）

目标：对压缩后的模型进行推理优化，实现异构硬件的高效部署。

任务 3.2.1 计算图优化（基于 ONNX）

1.模型转换与优化：

- 将实验 2 训练的模型（或压缩后模型）转换为 ONNX 格式，优化计算图。

2.优化效果评估：

- 用 ONNX Runtime 测试推理耗时，对比 PyTorch 原生推理（目标提速 20%+）；

- 用实验 1 的测试集验证精度一致性（误差≤0.5%）。

任务 3.2.2 硬件加速与跨平台部署

1.GPU 加速（TensorRT）

2. CPU 加速（OpenVINO）

3.跨平台部署验证：

- 在 GPU（TensorRT）和 CPU（OpenVINO）上分别运行优化模型；

- 用实验 1 的测试集（图像/文本）对比不同硬件的推理速度与精度。

五、重点与难点说明

重点关注

1.量化误差分析：

- 量化误差来源：权重与激活值的离散化损失，需用实验 1 的校准数据（训练集子集）优化量化参数（如动态范围）；

- 评估指标：计算量化前后模型在实验 1 测试集上的精度下降幅度（ΔAccuracy），并分析误差集中的类别（如实验 1 中“小目标图像”量化误差可能更高）。

2.精度与速度的权衡：

- 量化策略选择：CV 模型优先静态量化（精度更高），NLP 模型优先动态量化（速度提升更明显）；

- 剪枝比例控制：通过实验确定最优剪枝率（如 ResNet-18 剪枝 20%-30%时精度损失≤3%，速度提升≥25%）。

难点突破

1.量化后性能损失控制：

- 校准优化：增加校准数据量（实验 1 训练集前 500 样本），或采用混合精度量化（权重 INT8+激活 FP16）；

- 微调恢复：对量化/剪枝后的模型用实验 1 的训练集小批量微调（1-3 个 epoch），恢复精度损失。

2.异构硬件协同加速：

- 任务拆分：将实验 1 的多模态推理任务拆分（图像预处理→CPU，模型推理→GPU）；

- 动态调度：用 ONNX Runtime 自动选择最优执行 provider（如 GPU 可用时优先 TensorRT，否则用 CPU）。

六、实验报告要求

1. 说明小组成员构成，明确每个成员完成的任务；

2.模型优化对比分析：

- 量化/剪枝/蒸馏前后的参数量、计算量、模型大小对比表；

- 基于实验 1 测试集的精度损失分析（含误差分布热力图）。

3.推理加速效果验证：

- 不同优化方法（ONNX/TensorRT/OpenVINO）的速度提升对比；

- 异构硬件部署的精度一致性验证（与实验 2 原模型对比）。

4.关联分析：

- 分析实验 1 数据特点（如样本多样性）对量化误差的影响；

- 总结实验 2 模型特性（如 ResNet 的残差结构）对剪枝敏感性的影响。

七、参考资料

1. [PyTorch 量化文档]

(https://pytorch.org/docs/stable/quantization.html)

2. [ONNX 模型优化指南]

(https://onnxruntime.ai/docs/performance/)

3. [TensorRT 部署最佳实践]

(https://docs.nvidia.com/deeplearning/tensorrt/latest/performance/best-practices.html)

4. [OpenVINO 模型优化教程]

(https://docs.openvino.ai/latest/openvino_docs_model_optimization_guide.html)